<table>
<tr><td colspan="2">Nombre: Emmanuel Carrillo Loera

Edgar Daniel del Ángel Hernández

Fatima Alejandra Cavazos Serrato

Equipo: 3</td><td>Matrícula: 2067988

2094613

2099488</td></tr>
<tr><td>Nombre del curso: Componentes de Lógica de Negocios</td><td colspan="2">Nombre del profesor: Miguel Ángel Gómez Marroquín</td></tr>
<tr><td>Modulo Netacad: 3</td><td colspan="2">Actividad: ADR de actividad fundamental 3</td></tr>
<tr><td colspan="3">Fecha: 13/10/2026</td></tr>
<tr><td colspan="3">Bibliografía (Formato APA)

Archivo “Actividad_Unidad_3_CLN_ArquitecturaAlumnos.docx” proporcionado en la plataforma TEAMS

Archivo “03_Sazón_Urbano.docx” proporcionado en la plataforma TEAMS</td></tr>
</table>

ADR

ADR-01 Domain Model como patrón principal

Campo	Contenido
Título	Domain Model como patrón de lógica de negocio, con Transaction Script solo para marcar productos agotados
Contexto	Sazón Urbano maneja 15 reglas de negocio, 6 roles y 3 canales de pedido. Varias reglas dependen del estado del pedido por estación (RN-02, RN-03, RN-12) y deben aplicarse igual sin importar el canal. Las promociones cambian con frecuencia (por ejemplo, el 2x1 de martes que hoy se aplica “de memoria”)
Alternativas	Transaction Script: rápido de construir, pero duplicaría las reglas en cada flujo y cada cambio tocaría varios scripts. Table Module: muy bueno para reportes, pero no modela el ciclo de vida del pedido. Domain Model: reglas y estado juntos, a costa de más estructura inicial
Decisión	Domain Model como patrón principal. Transaction Script solo en CU-04a (marcar agotado), por ser una operación lineal. Table Module queda descartado en los 5 casos de uso
Consecuencias	Beneficios: una regla vive en un solo lugar y los cambios quedan localizados. Costos: más clases y estructura que con scripts. Riesgos: `Pedido` concentra mucho comportamiento (observación de la revisión SOLID). Trabajo posterior: los reportes (US-13 y US-17) podrían usar Table Module
Trazabilidad	NEC-04, NEC-05, NEC-11, NEC-12 · RN-01 a RN-06, RN-12, RN-14 · US-03, 05, 06, 07, 08, 09, 18 · CU-01 a CU-05

El sistema de *Sazón Urbano* presenta un nivel de complejidad de negocio considerable, abarcando 15 reglas de negocio, 6 roles operativos bien diferenciados y 3 canales de recepción de pedidos (mesa, teléfono/WhatsApp y domicilio). Un aspecto crítico del dominio es que múltiples reglas dependen estrictamente del estado dinámico del pedido por cada estación de cocina (por ejemplo, la regla RN-02 que prohíbe la cancelación sin autorización si ninguna estación ha comenzado a preparar, o la RN-03 que exige la autorización del Encargado y la generación de merma si la preparación ya dio inicio). Además, estas reglas deben cumplirse de manera idéntica sin importar el canal de origen, y las promociones operan bajo condiciones dinámicas y frecuentes (como el descuento de 2x1 en martes).

Para resolver esto, el equipo evaluó tres alternativas principales:

1. **Transaction Script:** Aunque es un enfoque sumamente rápido de construir para sistemas pequeños o lineales, se descartó para el flujo principal porque duplicaría las reglas de validación de estado en cada flujo, provocando que cualquier modificación menor obligara a tocar múltiples scripts de manera propensa a errores.
2. **Table Module:** Si bien es altamente eficiente para sistemas orientados a reportes masivos o consultas tabulares directas, carece por completo de la capacidad para modelar el ciclo de vida, las transiciones de estado y las invariantes de un pedido.
3. **Domain Model:** Permite encapsular las reglas de negocio, los datos y el comportamiento directamente dentro de los objetos del dominio (agregados)

Se seleccionó **Domain Model** como el patrón principal de lógica de negocio para proteger las invariantes del sistema. Como excepción justificada, se autorizó el uso de **Transaction Script** únicamente para el caso de uso CU-04a (*marcar un producto agotado por parte de cocina*), ya que se trata de una operación estrictamente lineal que no involucra reglas complejas de ciclo de vida.

Como principal beneficio, las reglas de negocio residen en un único lugar centralizado (dentro del aggregate Pedido), localizando cualquier cambio futuro. Como costo, se asume una mayor estructura inicial de clases y objetos.

ADR-02 Repository y Unit of Work

Campo	Contenido
Título	Repositorios con interfaz en el dominio y una Unit of Work por solicitud
Contexto	El dominio no debe depender de SQL ni de la base de datos. La cancelación con preparación iniciada debe guardar `Pedido` y `Merma` juntos (RN-03). El BRD advierte que la red puede tener intermitencias. No hay base de datos real en este ejercicio
Alternativas	Servicios con SQL directo: acopla el negocio a la tecnología. Repository sin Unit of Work: riesgo de guardar solo una parte de los cambios. Repository + Unit of Work por solicitud**:** más piezas, pero con atomicidad
Decisión	Interfaces de Repository en el dominio (9 métodos en total) e implementaciones en infraestructura (en memoria para el código base). Una Unit of Work por solicitud**:** se confirma al final o se deshace todo, y los eventos se publican después de confirmar. No se implementa protección contra concurrencia, para mantener el diseño simple
Consecuencias	Beneficios**:** se puede cambiar de tecnología sin tocar el negocio, y `cancelarPedido` nunca deja un pedido cancelado sin su merma. Costos**:** más interfaces. Riesgos**:** sin control de concurrencia, dos usuarios podrían actuar sobre un estado desactualizado (por ejemplo, cancelar mientras cocina inicia una estación); se acepta por ser un sistema ficticio de una sola base. Trabajo posterior: si pasara a producción, agregar control de versiones sobre el pedido
Trazabilidad	NEC-01, NEC-04 · RN-03, RN-13 · Restricciones del BRD (red intermitente) · Hallazgos H4 y H5 de la revisión SOLID

El análisis del negocio exige que operaciones complejas (como la cancelación de un pedido con preparación iniciada) garanticen atomicidad absoluta al registrar simultáneamente el estado del Pedido cancelado y la Merma generada (RN-03). Adicionalmente, el documento de requerimientos (BRD) advierte sobre posibles intermitencias en la red operativa.

Se diseñaron contratos e interfaces de repositorio puros ubicados en la capa de dominio (hablando exclusivamente en el lenguaje del negocio, con un total de 9 métodos) y sus respectivas implementaciones concretas en la capa de infraestructura (en memoria para el código base actual). Se implementó el patrón **Unit of Work por solicitud**, donde cada transacción se inicializa al comenzar la ejecución del *Handler*, se confirma al finalizar con éxito, o se revierte (*rollback*) por completo si se produce cualquier excepción. Como regla estricta, los eventos de dominio se publican únicamente después de que la unidad de trabajo ha sido confirmada exitosamente. Por simplicidad académica, se acordó no implementar mecanismos complejos de control de concurrencia explícita.

ADR-03 CQRS básico

Campo	Contenido
Título	CQRS básico con Handlers, DTO de lectura y una sola base de datos
Contexto	Hay operaciones que cambian el estado (registrar, cancelar, asignar) y necesidades de solo lectura (resumen del pedido, menú disponible, promociones vigentes, repartidores disponibles). US-04 pide que lo agotado “deje de mostrarse” sin meter una regla extra al registrar pedidos. La guía no exige bases de datos separadas
Alternativas	Un solo modelo para leer y escribir: expondría el aggregate `Pedido` a pantallas y mezclaría responsabilidades. CQRS completo con bases separadas: complejidad que un restaurante único no justifica. CQRS básico: separación en el código, una sola base
Decisión	9 Commands y 4 Queries, cada uno con su Handler. Las consultas dependen de interfaces de lectura y devuelven DTO, nunca el aggregate. Una sola base de datos
Consecuencias	Beneficios**:** responsabilidades claras, lecturas con el modelo adecuado y cumplimiento del principio I de SOLID. Costos**:** más clases (un handler por operación). Riesgos**:** duplicar conceptos entre el DTO y el aggregate. Trabajo posterior**:** los reportes pesados podrían separarse más adelante
Trazabilidad	NEC-03, NEC-08, NEC-09 · RN-05 · US-04, US-13, US-17 · Hallazgo H6

El sistema opera bajo una dualidad operativa clara: por un lado, ejecuta operaciones de escritura y modificación de estado (tales como registrar pedidos, aplicar promociones, cancelar pedidos o asignar repartidores); por otro lado, soporta requerimientos intensivos de solo lectura (como consultar el resumen detallado de un pedido, visualizar el menú disponible, verificar las promociones vigentes o listar los repartidores disponibles en tiempo real). Historias de usuario específicas, como la US-04, exigen que los productos marcados como agotados desaparezcan automáticamente de las consultas sin necesidad de inyectar reglas complejas adicionales dentro del flujo de registro de nuevos pedidos. Dado el contexto del sistema (un único restaurante), no se justifica una arquitectura de infraestructura distribuida con bases de datos físicas separadas.

Se adoptó un enfoque de **CQRS básico** estructurado mediante 9 *Commands* (para las acciones de escritura y modificación) y 4 *Queries* (para las operaciones exclusivas de lectura), operando cada uno a través de su respectivo *Handler* especializado. Las consultas de lectura dependen de interfaces dedicadas (ConsultasDe...) y retornan exclusivamente objetos de transferencia de datos planos (**DTOs**), evitando estrictamente la exposición de los agregados del dominio hacia las vistas. Se mantiene una única base de datos subyacente, y la acción de marcar un producto agotado se maneja como un comando simple sin read model independiente.

Los beneficios obtenidos son una separación nítida de responsabilidades en el código, consultas altamente eficientes orientadas a la interfaz de usuario mediante DTOs planos, y el cumplimiento estricto del principio de segregación de interfaces (I de SOLID). El costo principal es el aumento en el número de clases del proyecto (un *handler* por cada operación de comando o consulta).